

Gestión de datos de países en Python

Tecnicatura en Programación

Programación 1

Peruzzi, Facundo

Gómez, Gabriel

22/06/2026

Indice

Marco Teórico.....	3
Listas.....	3
Diccionarios.....	4
Funciones.....	4
 Decisiones Técnicas y Arquitectura.....	5
Diseño de la estructura de datos.....	5
Flujo.....	5
 Dificultades y conclusión.....	6
 Referencias Bibliográficas.....	7

Marco Teórico

El presente proyecto consiste en el desarrollo de una aplicación en Python destinada a gestionar información sobre países. El sistema permite cargar datos desde un archivo CSV, almacenar la información mediante estructuras de datos, realizar búsquedas, filtros, ordenamientos y generar estadísticas básicas.

Para su desarrollo se aplican conceptos fundamentales trabajados en Programación 1, como listas, diccionarios, funciones, estructuras condicionales, manejo de archivos y procesamiento de información.

La consigna establece como objetivo desarrollar una aplicación capaz de gestionar datos de países utilizando estas herramientas, aplicando modularización, validaciones y lectura desde archivos CSV.

Listas

Las listas son estructuras de datos que permiten almacenar una colección ordenada de elementos. En Python son estructuras mutables, es decir, que sus elementos pueden modificarse durante la ejecución del programa.

Dentro de este proyecto, las listas se utilizan principalmente durante la lectura del archivo CSV. La función encargada de cargar los países lee cada línea del archivo y utiliza el método `.split(",")` para separar los valores contenidos en esa línea, generando una lista temporal con los datos individuales del país.

Por ejemplo, una línea del archivo:

```
Argentina,America,10000,850000
```

al utilizar:

```
datos = linea.strip().split(",")
```

se transforma en:

```
["Argentina","45376763","2780400","América"]
```

Esta lista permite acceder a cada elemento según su posición para luego convertir esa información a la estructura utilizada por el programa.

Además, las listas son utilizadas en operaciones auxiliares como recorrer datos, almacenar resultados de búsquedas o filtros y trabajar con conjuntos temporales de información.

Diccionarios

Los diccionarios son estructuras de datos que almacenan información mediante pares de clave y valor. Cada clave representa un identificador que permite acceder al dato asociado de forma directa.

En este proyecto, la estructura principal utilizada para almacenar los países está basada en diccionarios anidados.

Cada país se almacena utilizando su nombre como clave principal, y dentro de esa clave se encuentra otro diccionario con sus características:

```
países = [{"Argentina": {"continente": "América", "poblacion": 45376763, "superficie": 2780400}}]
```

Esta decisión permite acceder rápidamente a la información de un país mediante su nombre, facilitando operaciones como:

- búsqueda por nombre.
- actualización de población y superficie.
- filtros.
- generación de estadísticas.

El uso de diccionarios anidados permite mantener los datos organizados y relacionados, evitando depender de posiciones numéricas como ocurre en una lista.

Funciones

Las funciones permiten dividir el programa en bloques independientes de código que realizan tareas específicas.

En este proyecto se utilizan funciones para separar las distintas responsabilidades del sistema, logrando un código más organizado y fácil de mantener.

Algunas funciones principales son:

- cargar_paises() → lee el archivo CSV y carga la información en la estructura de datos.
- Función para agregar países.
- Función para actualizar datos.
- Función para buscar países.
- Funciones para aplicar filtros.
- Funciones para ordenar información.
- Función para calcular estadísticas.

Esta organización permite que cada parte del programa tenga una responsabilidad determinada, evitando repetir código y facilitando la corrección de errores.

Ejemplo de Función:

```
def guardar_paises(paises):
    with open("paises.csv", "w", encoding='utf-8') as archivo:
        archivo.write("nombre,continente,poblacion,superficie\n")
        for nombre, info in paises.items():
            archivo.write(f"{nombre},{info['continente']},{info['poblacion']},{info['superficie']}\n")
```

Decisiones Técnicas y Arquitectura

Diseño de la estructura de datos

Para el almacenamiento principal de los datos se decidió utilizar un sistema basado en diccionarios anidados.

La estructura general tiene como clave principal el nombre del país, y como valor un diccionario interno que almacena sus atributos:

```
paises = {"España": {"continente": "Europa", "poblacion": 3000, "superficie": 90000}}
```

Esta estructura fue seleccionada porque permite acceder directamente a un país sin recorrer todos los registros, utilizando su nombre como identificador.

Proceso de carga de datos desde CSV

El archivo CSV funciona como fuente inicial de información.

La función cargar_paises() realiza el siguiente proceso:

1. Abrir el archivo CSV.
2. Leer cada línea.
3. Separar los datos utilizando .split(",").
4. Obtener una lista temporal con los valores.
5. Convertir esos valores al formato necesario.
6. Guardarlos dentro del diccionario principal.

Flujo:

Archivo csv



Leer línea



.split(",")



Lista temporal de datos

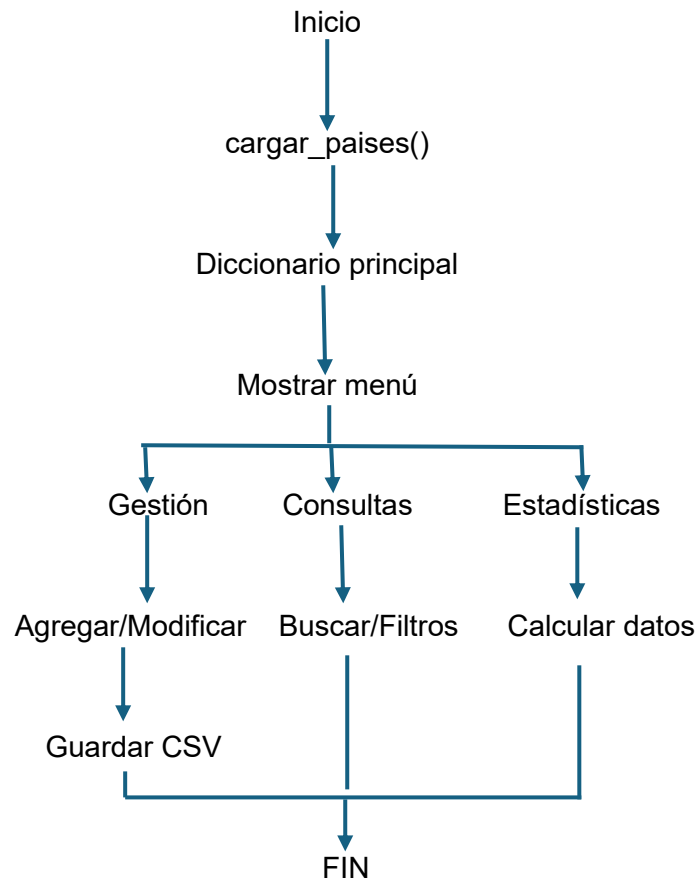


Crear diccionario del país



Guardar en el diccionario principal

Arquitectura general del sistema



Dificultades y conclusiones:

En términos generales, el desarrollo del sistema no presentó dificultades significativas, ya que la implementación de las funcionalidades principales resultó comprensible y pudo llevarse a cabo de manera organizada. Sin embargo, durante el proceso de desarrollo surgieron distintos errores y ajustes necesarios, los cuales fueron solucionados progresivamente, tal como se refleja en el historial de commits del proyecto.

Uno de los principales desafíos encontrados fue la resolución de un problema relacionado con las coincidencias parciales dentro de la función `buscar_pais()`, donde fue necesario analizar el funcionamiento de la búsqueda y ajustar la lógica para obtener resultados correctos. Además, otro punto que requirió atención fue el proceso de integración del código mediante ramas, especialmente al realizar el *merge* entre versiones y resolver manualmente los conflictos generados.

A pesar de estas dificultades, la experiencia resultó altamente enriquecedora, ya que el desarrollo en conjunto permitió conocer distintas formas de abordar un mismo problema. Debido a que cada integrante posee una forma diferente de organizar y escribir código, el trabajo colaborativo permitió intercambiar ideas, aprender nuevas estrategias de resolución y mejorar la comprensión de los conceptos aplicados.

También fue una oportunidad para incorporar nuevas herramientas y conocimientos, como el uso de funciones anónimas mediante lambda, que permiten realizar operaciones específicas de manera más eficiente sin necesidad de definir una función tradicional utilizando def.

En conclusión, el proyecto permitió aplicar los conocimientos adquiridos durante la materia en una situación práctica, fortaleciendo tanto los aspectos técnicos de programación como las habilidades de trabajo colaborativo. El desarrollo del sistema representó una experiencia positiva que aportó nuevos aprendizajes y permitió comprender mejor el proceso completo de creación, depuración y mejora de una aplicación.

[GitHub](#)

[Youtube](#)

Referencias bibliográficas

Tecnicatura Universitaria en Programación. (s.f.). *2 - Apertura, lectura, escritura, cierre de archivos*. Material de estudio proporcionado por la cátedra de Programación 1.

Tecnicatura Universitaria en Programación. (s.f.). *3 - strip(), split(), join()*. Material de estudio proporcionado por la cátedra de Programación 1.

Tecnicatura Universitaria en Programación. (s.f.). *4 - Archivos CSV (listas y diccionarios)*. Material de estudio proporcionado por la cátedra de Programación 1.

Tecnicatura Universitaria en Programación. (s.f.). *5 - Funciones*. Material de estudio proporcionado por la cátedra de Programación 1.

Tecnicatura Universitaria en Programación. (s.f.). *Alcances*. Material de estudio proporcionado por la cátedra de Programación 1.

Tecnicatura Universitaria en Programación. (s.f.). *Listas - Actividad 5*. Material de estudio proporcionado por la cátedra de Programación 1.

Tecnicatura Universitaria en Programación. (s.f.). *Listas Actividad 4*. Material de estudio proporcionado por la cátedra de Programación 1.

Tecnicatura Universitaria en Programación. (s.f.). *Actividad 2 - Listas*. Material de estudio proporcionado por la cátedra de Programación 1.

Tecnicatura Universitaria en Programación. (s.f.). *Manejo de errores con archivos*. Material de estudio proporcionado por la cátedra de Programación 1.

Tecnicatura Universitaria en Programación. (s.f.). *Manejo de errores: Tipo de errores*. Material de estudio proporcionado por la cátedra de Programación 1.

Tecnicatura Universitaria en Programación. (s.f.). *Manejo de errores: Try Except*. Material de estudio proporcionado por la cátedra de Programación 1.